

Архитектурное описание библиотеки количественного финансового анализа (QuantLib)

Целевая аудитория: Разработчики

1 Контекст и назначение системы

Разрабатываемая система представляет собой высокопроизводительную библиотеку для численного моделирования финансовых процессов (в частности, ценообразования опционов по модели Блэка-Шоулса).

2 Структура системы

Архитектура построена на основе **Layered style**.

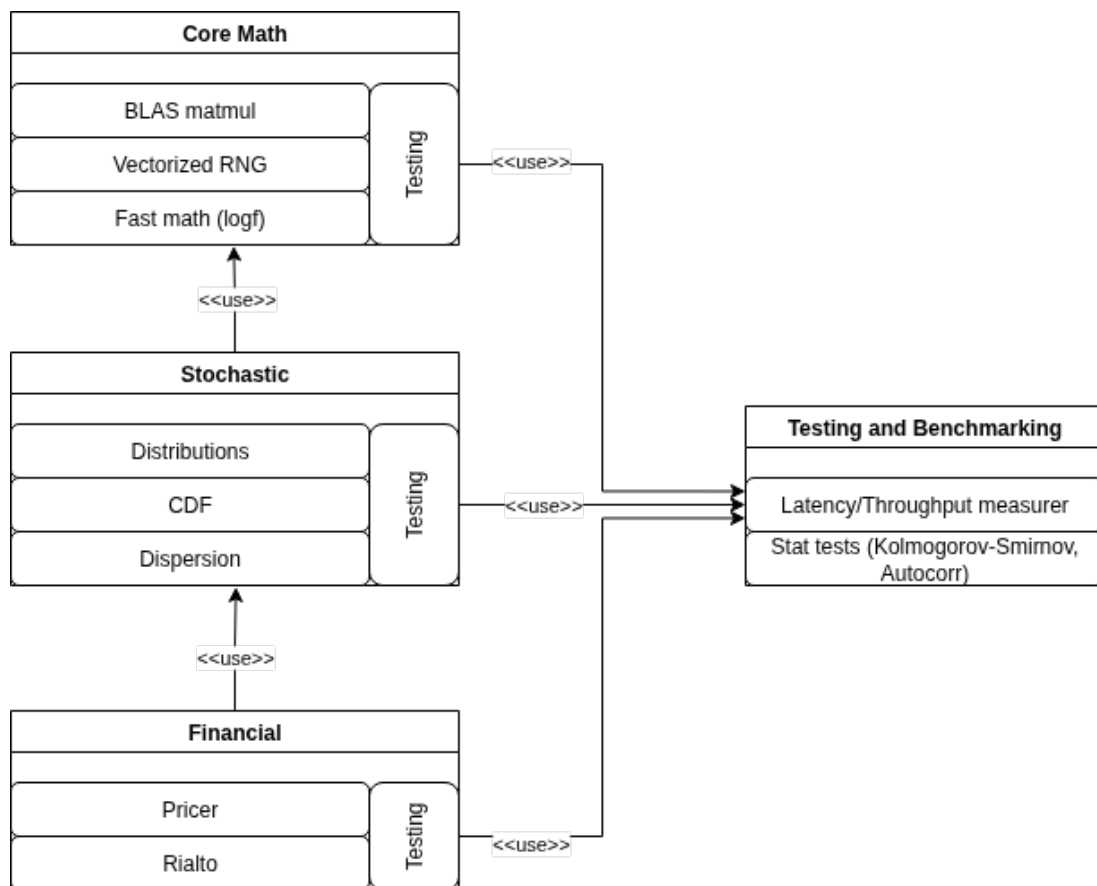


Диаграмма пакетов разрабатываемой системы

Система декомпозирована на следующие модули:

- **Core Math (C/C++, SIMD):** Слой платформозависимых оптимизаций. Содержит векторизованный (AVX-512) генератор `MinstdRandVec`, оптимизированное перемножение матриц с учетом кэш-локальности и кастомную функцию `logf` с точностью 3ULP.
- **Stochastic (C++):** Алгоритмический слой. Предоставляет генераторы из различных распределений (Normal, Gamma, ChiSquare), функции CDF и устойчивые к потере точности алгоритмы расчета дисперсии (`Disp1Pass`, `Disp2Pass`).
- **Financial (C++):** Бизнес-логика. Реализует прайсинг опционов (модули `Rialto` и `Pricer`) аналитическим методом и методом Монте-Карло с поддержкой пакетной обработки.
- **Testing and Benchmarking:** Содержит статистические тесты (Колмогорова-Смирнова, автокорреляция) и фреймворк `Runner` для замеров latency и throughput через `rdtscp` с защитой от out-of-order execution.

3 Обоснование архитектурных решений

3.1 Разделение на вычислительное ядро и бизнес-логику:

Финансовые модули изолированы от деталей работы с SIMD-инструкциями и аппаратными таймерами. Это позволяет разработчикам бизнес-логики оперировать высокоуровневыми концепциями, в то время как Core-разработчики могут менять реализацию `matmul` или RNG без влияния на верхний слой.

3.2 Использование C для ядра и C++ для верхних уровней:

Низкоуровневые вычисления реализованы на C с использованием `restrict` указателей для максимизации агрессивных оптимизаций компилятора. Статистический слой использует шаблоны C++ для обеспечения zero-cost абстракций над числами с плавающей точкой.

3.3 Внедрение независимого инфраструктурного слоя Benchmarking:

Система имеет критические нефункциональные требования к производительности. Поэтому профилирование вынесено в отдельный архитектурный компонент. Использование `taskset`, изоляции ядер и `cgroup` governor на уровне Makefile гарантирует, что разработчики будут получать воспроизводимые метрики, валидируя дизайн-требования к latency и throughput.